

Verified Transformations on Demand for the λ_{ANF} Pipeline

Master’s Thesis Proposal

Advisor: Zoe Paraskevopoulou

Motivation. Adding an optimization to a verified compiler requires both an implementation and a proof that it preserves program behavior. CertiRocq’s (formerly CertiCoq’s) λ_{ANF} middle end provides a promising setting for reducing this effort: several transformations share an intermediate language and a logical-relations framework for compositional correctness [1]. The recent ANF experience report [2] demonstrated how an LLM, guided by a human expert, could adapt an existing CPS correctness proof to a related ANF transformation. This thesis will investigate whether the same approach can support the development of new optimizations within the λ_{ANF} pipeline.

Aim and research question. The aim is to explore *verified transformations on demand*: given a description of a desired optimization, its side conditions, and a suitable existing proof, use an LLM to help construct a reusable compiler pass and its Rocq correctness proof. The central question is: *how far can existing λ_{ANF} proofs serve as templates for new transformations, and what human reasoning remains necessary?* Here, “on demand” means developing a new verified pass in response to an optimization need; the resulting theorem must cover every input satisfying its stated preconditions.

Candidate transformations. The main case study will be *function specialization*: generating versions of recursive functions that exploit information about their arguments. The inspected λ_{ANF} pipeline already provides inlining, uncurrying, lambda lifting, and dead-parameter elimination [4]; specialization would make additional information available throughout a recursive computation.

1. **Specialization for known function arguments (core).** A call `map inc xs` can be redirected to a new recursive function `map_inc xs`. Its body calls `inc` directly and recurses through `map_inc`, exposing opportunities for inlining and eliminating the repeatedly passed function argument. Schematically, using source-like syntax:

```
map h (Cons x xs) = Cons (h x) (map h xs)
map_inc (Cons x xs) = Cons (inc x) (map_inc xs)
```

The empty-list branch is unchanged. The original `map` remains available for other calls. The function parameter is live in the original body, so dead-parameter elimination alone cannot achieve this result; specialization exposes the known callee at every recursive iteration. The initial implementation will handle singly recursive functions and one statically known function argument without captured data, in scope where the variant is defined. Matching recursive calls will reuse the variant; other calls will retain the generic function. A bound on generated variants will control code growth and ensure the transformation terminates.

2. **Constructor-argument specialization (extension).** Following the idea of call-pattern specialization [3], generate a variant such as `loop_pair a b` for calls `loop (Pair a b)`. The variant receives fields directly and reconstructs the aggregate if needed. Existing shrinking can remove redundant reconstruction, projections, and case analysis, including within recursive loops. This case study would extend the specialization infrastructure from known functions to known data shapes. It will initially target one constructor layer and retain the generic entry point for other argument shapes.

Methodology. The lambda-lifting and dead-parameter-elimination proofs will provide templates for relating functions with different parameter lists and redirecting calls. Inlining proofs will supply patterns for substitution and fresh renaming; shrinking proofs will support simplification of specialized bodies. The key new invariant must relate a generic function applied to the selected static arguments to its specialized recursive variant. Establishing and maintaining this invariant is the central proof challenge.

For each case study, the student will define a relational specification, an executable transformation, and a precise correctness statement. An LLM will receive the relevant proof templates and an explanation of their intended adaptation, then develop the implementation and proofs using Rocq feedback. The student will design invariants, review theorem statements, and record where mathematical guidance or manual proof work is required.

The proof obligations comprise correspondence between the implementation and its relational specification, preservation of scoping and unique bindings, and semantic preservation under the existing logical relation, including the resource conditions needed for divergence preservation.

Evaluation and expected contribution. Evaluation will address both compiler performance and proof-development effort. Performance measurements will use existing and new CertiRocq benchmarks, including recursive list and tree traversals with known function arguments. Comparisons against the existing pipeline will measure execution time, allocation, generated-code size, and compilation time, including cases where specialization is unprofitable. Logs will record success, human intervention time, proof repairs, and reused versus new lemmas.

The expected outcome is a verified function-specialization pass and a documented workflow for developing transformations on demand. The extension will test whether the resulting implementation and proof templates transfer to specialization on data shapes. The thesis will identify reusable proof structure, new invariants, and the limits of LLM-assisted adaptation.

Indicative six-month plan. Month 1: study the framework and fix specifications; months 2–3: implement and verify specialization for known functions; month 4: investigate constructor-argument specialization; month 5: integrate and evaluate; month 6: consolidate proofs, document limitations, and write the thesis. The first pass is the core deliverable; the extension depends on progress.

References

- [1] Z. Paraskevopoulou, J. M. Li, and A. W. Appel. *Compositional Optimizations for CertiCoq*. PACMPL 5 (ICFP), Article 86, 2021. doi:10.1145/3473591.
- [2] Z. Paraskevopoulou. *Machine-Generated, Machine-Checked Proofs for a Verified Compiler (Experience Report)*. PACMPL 10 (ICFP), Article 302, 2026. doi:10.1145/3828700.
- [3] S. Peyton Jones. *Call-pattern Specialisation for Haskell Programs*. ICFP, 2007. Author’s paper.
- [4] CertiRocq contributors. *CertiRocq source code*, `theories/LambdaANF`, revision `f050f1f8`. Source and correctness proofs.